PROGRESSIVE FEATURE ENGINEERING AND IMBALANCED MACHINE LEARNING FOR CREDIT CARD FRAUD DETECTION

BY

[Student Name]

Matriculation Number: [Matriculation Number]

A RESEARCH PROJECT SUBMITTED TO

[Department]

[Faculty]

UNIVERSITY OF IBADAN

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF

[Degree Programme]

SUPERVISOR: [Supervisor Name]

DATE: [Month Year]

CERTIFICATION PAGE

This page is reserved for the certification statement, supervisor endorsement, and required signatures after review by the supervisor and department.

DEDICATION PAGE

This page is reserved for the dedication statement to be completed by the student before final submission.

ACKNOWLEDGMENT PAGE

This page is reserved for acknowledgments to the supervisor, department, family, colleagues, and other contributors as appropriate before final submission.

ABSTRACT

Credit card fraud detection is an important binary classification problem because fraudulent transactions are usually less frequent than legitimate transactions but have significant financial and operational consequences. This study developed a reproducible tabular machine-learning workflow using a researcher-generated synthetic credit card transaction dataset. The cleaned master dataset contained 499,985 transactions, consisting of 62,500 fraudulent transactions and 437,485 legitimate transactions, with a fraud ratio of 12.5004%. Five progressively enriched dataset versions were constructed from the same transaction population in order to examine the effect of feature engineering without changing the underlying sample. The models evaluated were Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost. The experimental design used a stratified 60/20/20 train-validation-test split, 5-fold cross-validation on the training split, PR-AUC implemented as average precision for hyperparameter tuning, validation-based model-family selection, and untouched test-set reporting. SMOTE was evaluated only inside the training pipeline after preprocessing and inside cross-validation folds. At the default threshold, the best result was obtained by v3 SMOTE Random Forest, with precision 0.998525, recall 0.758080, fraud-class F1 of 0.861846, and PR-AUC of 0.855235. After validation-based threshold optimization, the best operating point was v5 No-SMOTE Random Forest at threshold 0.80, with precision 0.999156, recall 0.758000, and fraud-class F1 of 0.862030. The findings indicate that progressive feature engineering improved fraud-class performance and that the observed benefit of SMOTE was threshold-dependent. The report also emphasizes reproducibility, transparent dataset auditing, and cautious interpretation of small performance differences. The study is limited by the synthetic dataset, absence of external validation, and leakage-risk proxy indicators such as MerchantRisk and CardRisk.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ANN	Artificial Neural Network
AUC	Area Under the Curve
CV	Cross-Validation
F1	F1-score
FN	False Negative
FP	False Positive
ML	Machine Learning
PR-AUC	Precision-Recall Area Under the Curve
ROC-AUC	Receiver Operating Characteristic Area Under the Curve
SMOTE	Synthetic Minority Oversampling Technique
TN	True Negative
TP	True Positive
XGBoost	Extreme Gradient Boosting

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Digital payment channels generate large streams of transaction records. Within these records, fraudulent activity is often a minority event that must be detected without overwhelming legitimate transaction activity. This makes credit card fraud detection a suitable case for imbalanced binary classification, where the aim is not simply to obtain high accuracy but to identify fraud with useful precision and recall.

Machine learning has become an important approach for fraud analysis because it can learn statistical patterns from transaction-level attributes. However, the quality of the results depends heavily on dataset preparation, feature engineering, class-imbalance handling, model selection, and evaluation metrics. Figure 1.1 provides a conceptual view of the digital payment fraud context without making unsupported numerical claims.

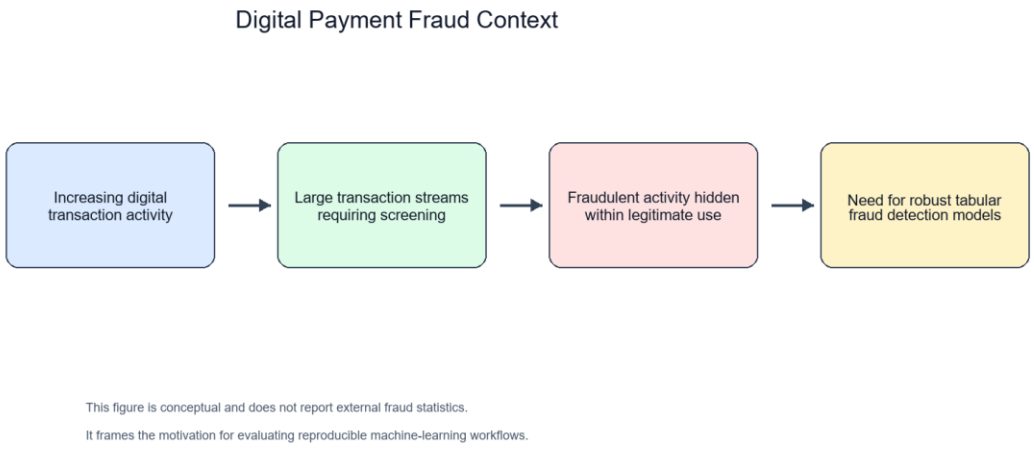

Figure 1.1: Conceptual digital payment fraud context and need for analytical detection.

This study focuses on a researcher-generated synthetic credit card transaction dataset. The work is therefore an academic and reproducibility-oriented investigation of tabular fraud classification, not a live banking deployment.

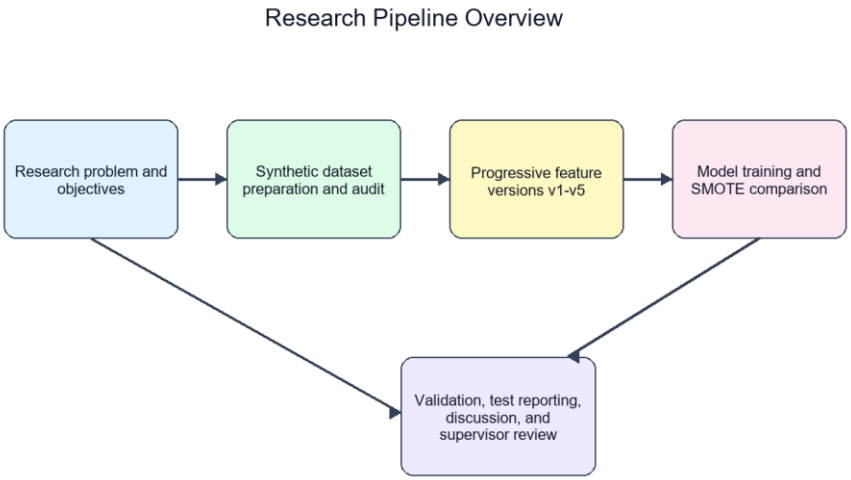

Figure 1.2: High-level research workflow for the study.

1.2 Statement of the Problem

Fraud detection research can be weakened by unclear dataset provenance, duplicated transactions, inconsistent feature sets, and overreliance on accuracy. In the project repository, legacy and merged data artifacts existed alongside newer cleaned datasets. A defensible supervisor-submission report therefore requires a clear statement of which datasets were used, how duplicates were handled, how features were progressively introduced, and how model performance was evaluated.

The problem addressed in this study is how to build and evaluate a reproducible tabular machine-learning pipeline for credit card fraud detection while controlling for dataset population, class imbalance, and possible leakage-risk features.

1.3 Aim and Objectives of the Study

The aim of the study is to evaluate the effect of progressive feature engineering and imbalanced machine-learning strategies on credit card fraud detection using a cleaned synthetic transaction dataset.

1. To prepare and validate a clean master transaction dataset for fraud classification.
2. To create five progressively enriched dataset versions from the same transaction population.

3. To train and compare Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost models.

4. To compare No-SMOTE and SMOTE training settings under a leakage-aware pipeline.

5. To evaluate default-threshold and threshold-tuned model performance using fraud-class metrics and PR-AUC.

1.4 Research Questions

1. How does progressive feature engineering affect fraud-class performance across v1 to v5?

2. How do No-SMOTE and SMOTE training settings compare at the default threshold?

3. Does threshold optimization change the practical advantage observed under SMOTE?

4. Which model families provide the strongest fraud-class performance on the cleaned synthetic dataset?

5. What limitations affect interpretation of the study results?

1.5 Significance of the Study

The study is significant because it demonstrates a reproducible workflow for preparing, validating, and evaluating progressively enriched fraud-detection datasets. It also shows why minority-class metrics, threshold policies, and transparent limitation statements are important in imbalanced fraud-detection research.

1.6 Scope of the Study

The study is limited to binary tabular classification on a researcher-generated synthetic credit card transaction dataset. It evaluates five dataset versions, five model families, No-SMOTE and SMOTE settings, and threshold optimization based on validation predictions. It does not evaluate live transaction streams, institutional data, or deployment infrastructure.

1.7 Limitations of the Study

The main limitations are the synthetic nature of the dataset, absence of external validation, possible leakage risk in MerchantRisk and CardRisk, static offline splitting, and the absence of repeated-seed or confidence-interval analysis.

1.8 Definition of Key Terms

FraudFlag: The binary target label indicating whether a transaction is fraudulent or legitimate.

SMOTE: A resampling technique that creates synthetic minority-class examples during training.

PR-AUC: A metric that summarizes the precision-recall relationship and is useful in imbalanced classification.

Feature engineering: The process of creating or transforming variables to improve predictive performance.

Threshold optimization: The process of selecting a probability cutoff based on validation predictions before test evaluation.

1.9 Organization of the Study

Chapter One introduces the study. Chapter Two reviews relevant literature. Chapter Three presents the methodology. Chapter Four reports and discusses the results. Chapter Five concludes the study and provides recommendations.

CHAPTER TWO: LITERATURE REVIEW

2.1 Conceptual Review

2.1.1 Credit Card Fraud

Credit card fraud involves unauthorized or deceptive use of payment-card information. Prior fraud-detection literature emphasizes that fraud patterns are dynamic and that detection systems must distinguish suspicious behavior from legitimate high-volume transaction activity [1], [2].

2.1.2 Fraud Detection Systems

Fraud detection systems traditionally use business rules, thresholds, manual reviews, and risk indicators. Rule-based systems are interpretable but may require frequent updating when fraudulent behavior changes [1], [2].

2.1.3 Machine Learning in Fraud Detection

Machine-learning systems estimate fraud risk by learning relationships between transaction attributes and labels. Prior studies have used linear classifiers, tree-based methods, ensemble learning, and boosting methods for fraud detection [3]-[8].

2.1.4 Imbalanced Classification

Fraud detection is usually imbalanced because fraudulent transactions occur less frequently than legitimate transactions. Imbalanced learning literature recommends focusing on minority-class performance, resampling strategies, and cost-sensitive evaluation [9]-[14].

2.1.5 Feature Engineering

Feature engineering transforms raw variables into more informative predictors. In fraud detection, transaction amount, time, merchant, card, category, and risk-derived indicators can influence model performance [5], [29], [30].

2.1.6 Evaluation Metrics

Accuracy is often insufficient in imbalanced classification because a model can perform well on the majority class while missing minority-class cases. F1, ROC-AUC, and PR-AUC are therefore commonly used to assess classifier behavior [15]-[17].

2.2 Theoretical Review

The study is grounded in supervised learning theory, where models learn a mapping from input features to a known target label. It also draws on imbalanced learning theory, which explains why class distribution affects model training and evaluation. Ensemble learning theory is relevant because Random Forest, XGBoost, and CatBoost combine multiple decision structures to improve predictive performance [18]-[22]. Finally, the risk-based fraud detection framework motivates the use of transaction and proxy risk indicators while requiring caution about feature provenance.

2.3 Empirical Review

Empirical studies in credit card fraud detection have shown that model performance depends on data preparation, feature construction, class imbalance handling, and evaluation design [3]-[8]. SMOTE and related methods are widely used to address minority-class scarcity, but their benefit depends on the model, data distribution, and operating threshold [9]-[14].

Tree-based and boosting models are often competitive on tabular fraud data because they can model nonlinear feature interactions [18]-[22]. Prior work also emphasizes careful model selection and validation to avoid overfitting and biased performance estimates [23]-[28].

2.4 Research Gap

The gap addressed in this study is the need for a controlled comparison of progressive feature engineering across the same cleaned transaction population. This design helps separate feature effects from changes caused by sample size, duplicate records, or class-ratio differences.

2.5 Summary of Literature Review

The literature supports the use of machine learning, feature engineering, and minority-class metrics in fraud detection. It also cautions that resampling, model selection, and proxy risk indicators must be handled carefully to avoid unsupported claims.

CHAPTER THREE: METHODOLOGY

3.1 Research Design

The study adopted an experimental quantitative design. Five dataset versions were prepared from the same cleaned master population and evaluated using the same train-validation-test protocol.

3.2 Dataset Generation and Description

The master dataset was data/research_master_dataset.csv. It contained 499,985 transactions, with 62,500 fraud cases and 437,485 legitimate cases. The target label was FraudFlag. Language-processing features were not present; the task was binary tabular classification.

Table 3.1: Dataset Summary

Property	Value
Dataset file	data/research_master_dataset.csv
Dataset type	Researcher-generated synthetic transaction dataset
Task	Binary tabular classification
Target label	FraudFlag
Total transactions	499,985
Fraud transactions	62,500
Legitimate transactions	437,485
Fraud ratio	12.5004%
Missing values in final datasets	0
Exact duplicate rows in final datasets	0
Duplicate TransactionID values in final datasets	0
Language-processing features	Not present

3.3 Dataset Cleaning and Validation

The raw merged provenance file had 849,999 rows and 350,014 exact duplicate rows. It was not used as an additional main experiment. The final datasets had zero missing values, zero exact duplicate rows, and zero duplicate TransactionID values.

3.4 Dataset Versioning Strategy

The five versions were designed to be progressively feature-rich while retaining the same transaction population. This improved experimental control because model changes could be interpreted in relation to feature availability.

Table 3.2: Dataset Versions

Version	Rows	Columns	Added Feature Group	Purpose
v1	499,985	8	Baseline transaction, categorical, and time fields	Baseline comparator
v2	499,985	11	High-amount flag, night-transaction flag, subcategory	Assess simple risk indicators
v3	499,985	13	Merchant-risk and card-risk indicators	Assess proxy risk enrichment
v4	499,985	21	Amount and time engineered features	Assess advanced engineered predictors
v5	499,985	23	Night-high-amount flag and composite risk score	Assess full engineered feature set

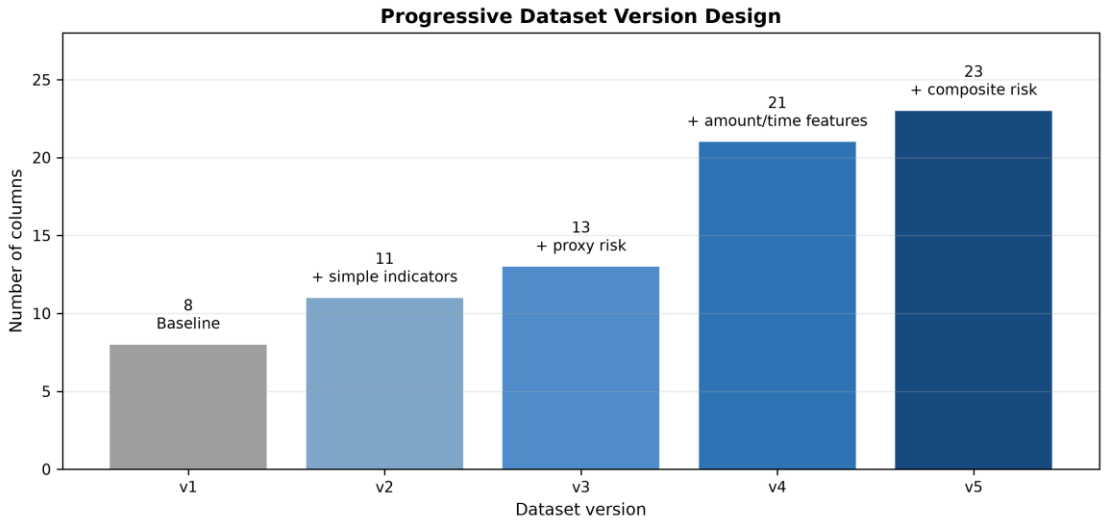

Figure 3.2: Progressive dataset version design from v1 to v5.

3.5 Feature Engineering

The feature engineering strategy introduced high-amount flags, night transaction indicators, subcategory information, merchant and card proxy risk indicators, amount transformations, time-derived features, and composite risk scores. MerchantRisk and CardRisk were treated cautiously because they may be target-proximal proxy features.

3.6 Data Preprocessing

Logistic Regression used imputation, one-hot encoding, and numeric scaling. Decision Tree, Random Forest, and XGBoost used imputation and frequency encoding for categorical variables. CatBoost followed the repository implementation, with preprocessing and feature handling integrated according to the final pipeline.

3.7 Train/Validation/Test Split

The split was stratified at a 60/20/20 ratio. The training set contained 299,991 records, the validation set contained 99,997 records, and the test set contained 99,997 records. Model selection used the validation split, while final reporting used the untouched test split.

3.8 Class Imbalance Handling with SMOTE

SMOTE was applied only inside the imblearn training pipeline, after preprocessing and inside cross-validation folds. It was not applied before the train-validation-test split.

3.9 Machine Learning Algorithms

3.9.1 Logistic Regression

Logistic Regression served as a linear baseline for tabular fraud classification.

3.9.2 Decision Tree

Decision Tree models created rule-based splits over preprocessed transaction features.

3.9.3 Random Forest

Random Forest used an ensemble of decision trees to reduce variance and capture nonlinear feature interactions.

3.9.4 XGBoost

XGBoost used gradient-boosted decision trees and was evaluated as a strong tabular classifier.

3.9.5 CatBoost

CatBoost was evaluated because of its suitability for datasets containing categorical variables and nonlinear relationships.

3.10 Hyperparameter Tuning

Hyperparameter tuning used 5-fold cross-validation on the training split, with PR-AUC implemented as average precision as the tuning metric.

3.11 Threshold Optimization

Threshold optimization selected probability cutoffs on validation predictions and then evaluated the selected cutoffs on test predictions.

3.12 Evaluation Metrics

The study reported fraud precision, fraud recall, fraud-class F1, ROC-AUC, and PR-AUC. Fraud-class F1 and PR-AUC were emphasized because the problem was imbalanced.

3.13 Experimental Pipeline

The complete workflow is shown in Figure 3.1. It includes dataset preparation, feature versioning, preprocessing, cross-validated tuning, validation selection, test evaluation, SMOTE comparison, and threshold tuning.

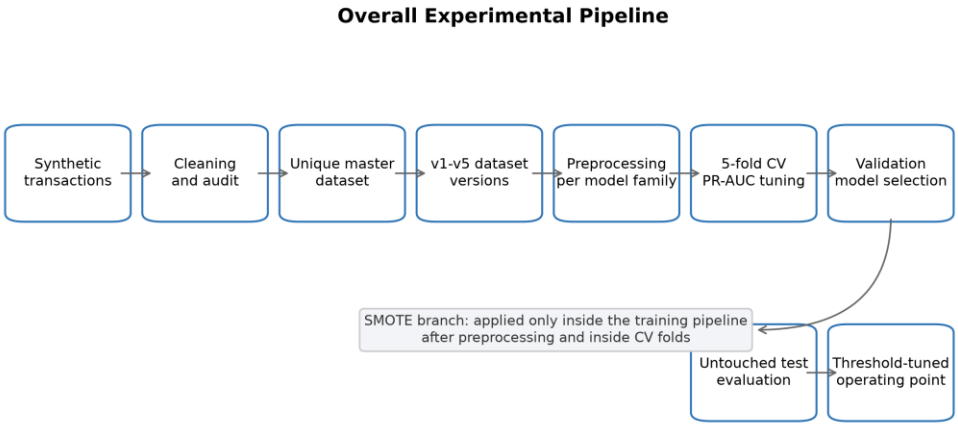

Figure 3.1: Experimental pipeline for dataset preparation, model tuning, evaluation, SMOTE comparison, and threshold optimization.

Table 3.3: Experimental Configuration

Component	Configuration
Split strategy	Stratified train/validation/test split
Split ratio	60/20/20
Training records	299,991
Validation records	99,997
Test records	99,997
Cross-validation	5-fold CV on training split
Hyperparameter tuning metric	PR-AUC / average precision
Model-family selection	Validation split
Final reporting	Untouched test split
SMOTE placement	Inside imblearn pipeline, after preprocessing, inside CV folds
Excluded model features	FraudFlag, TransactionID, Time, DayOfWeek, Month, IsWeekend, IsWeekendDerived

3.14 Tools and Software

The repository used Python scripts for dataset generation, validation, model training, threshold tuning, and reporting. The active scripts included scripts/generate_all_versions.py, scripts/ml_pipeline_core.py, and scripts/run_experiments_standard.py.

3.15 Ethical Considerations

The study used synthetic transaction data and did not use personally identifiable customer information or live bank records.

3.16 Reproducibility Measures

Reproducibility was supported through deterministic dataset files, synthetic TransactionID values, fixed dataset summaries, validation outputs, saved prediction files, and documented audit reports.

CHAPTER FOUR: RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents dataset audit results, class distribution, model performance under No-SMOTE and SMOTE settings, threshold optimization, feature importance, and answers to the research questions.

4.2 Dataset Audit Results

The final datasets passed validation checks for missing values, duplicate rows, and duplicate TransactionID values. The cleaned master dataset contained 499,985 unique transactions.

4.3 Class Distribution

The final class distribution consisted of 437,485 legitimate transactions and 62,500 fraudulent transactions.

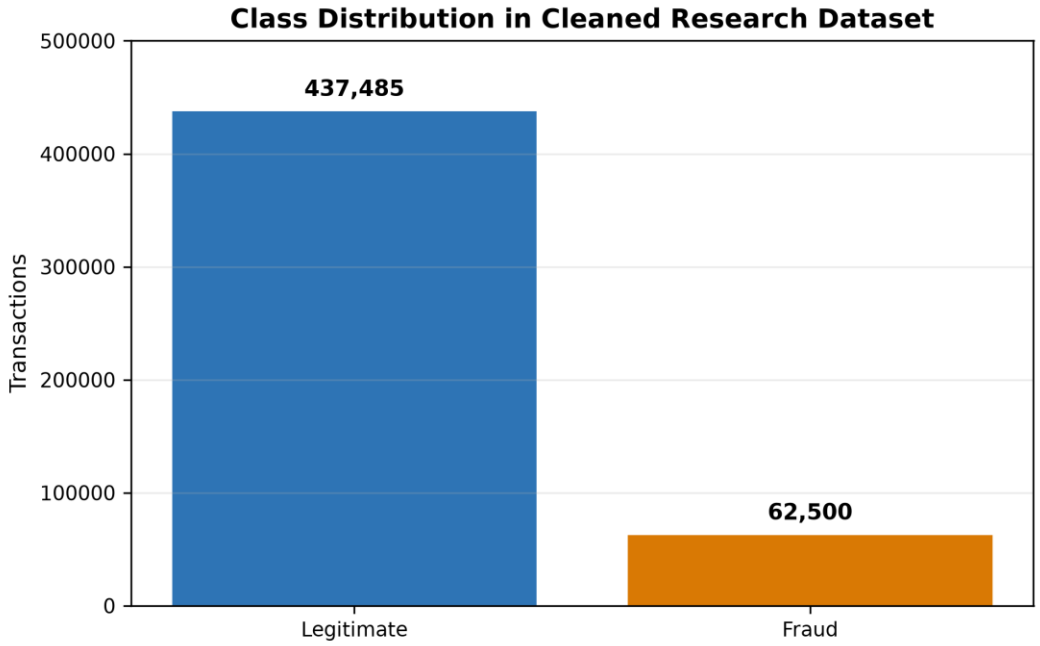

Figure 4.1: Class distribution of the cleaned synthetic transaction dataset.

4.4 Dataset Version Summary

All five versions used the same transaction population. The versions differed only by feature availability, from baseline transaction attributes in v1 to a full engineered feature set in v5.

4.5 No-SMOTE Model Results

Table 4.1 presents the selected default-threshold No-SMOTE results. Performance improved substantially after v2, with v3-v5 outperforming v1-v2.

Table 4.1: Best Default-Threshold Results Without SMOTE

Version	Best Model	Precision	Recall	Fraud-Class F1	PR-AUC
v1	XGBoost	0.8062	0.7502	0.7772	0.8268
v2	XGBoost	0.8094	0.7500	0.7786	0.8274
v3	CatBoost	0.9219	0.7739	0.8414	0.8558
v4	DecisionTree	0.9537	0.7678	0.8507	0.8337
v5	RandomForest	0.9418	0.7713	0.8480	0.8561

4.6 SMOTE Model Results

Table 4.2 presents the selected default-threshold SMOTE results. The best default-threshold result was v3 SMOTE Random Forest, with precision 0.998525, recall 0.758080, fraud-class F1 of 0.861846, and PR-AUC of 0.855235.

Table 4.2: Best Default-Threshold Results With SMOTE

Version	Best Model	Precision	Recall	Fraud-Class F1	PR-AUC
v1	DecisionTree	0.9760	0.7040	0.8180	0.8237
v2	RandomForest	0.9527	0.7153	0.8171	0.8265
v3	RandomForest	0.9985	0.7581	0.8618	0.8552
v4	CatBoost	0.9998	0.7570	0.8616	0.8564
v5	CatBoost	0.9999	0.7571	0.8617	0.8558

4.7 Comparative Analysis of No-SMOTE and SMOTE Results

SMOTE improved default-threshold fraud-class F1 across all five dataset versions. However, it did not improve recall in every selected model, showing that resampling affected the precision-recall trade-off rather than improving all metrics simultaneously.

Figure 4.2: Fraud-class F1 comparison between No-SMOTE and SMOTE results.

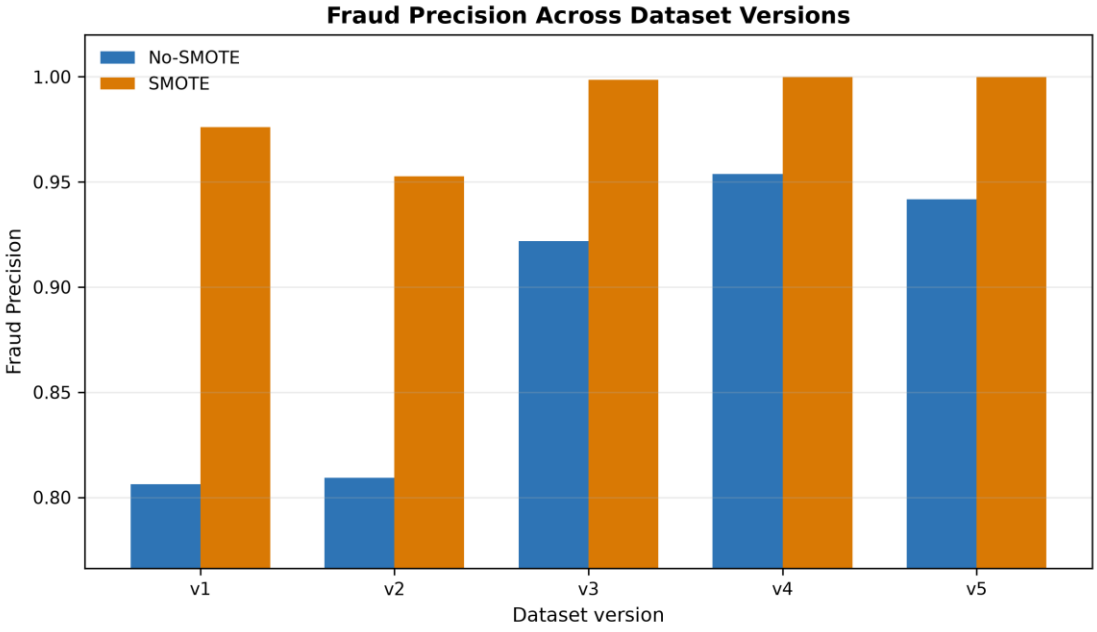

Figure 4.3: Fraud precision comparison between No-SMOTE and SMOTE results.

Figure 4.4: Fraud recall comparison between No-SMOTE and SMOTE results.

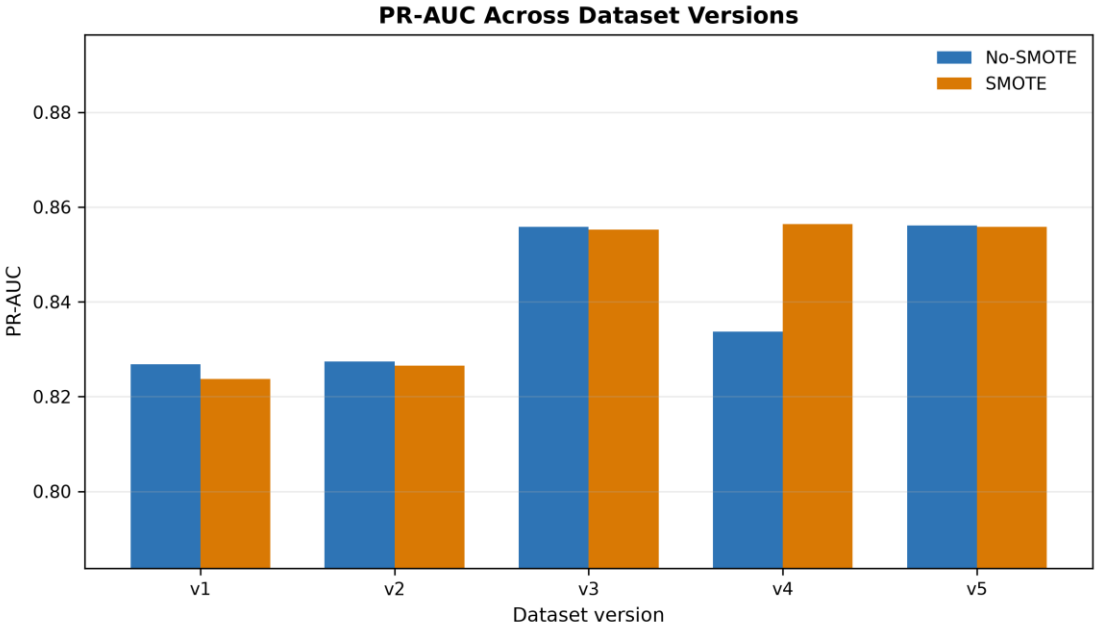

Figure 4.5: PR-AUC comparison between No-SMOTE and SMOTE results.

4.8 Impact of Progressive Feature Engineering

The largest performance gain occurred from v2 to v3, when merchant and card proxy risk indicators were introduced. The strongest v3-v5 results were practically close, so small differences should not be overstated without repeated-seed or uncertainty analysis.

4.9 Threshold Optimization Results

Table 4.3 presents the threshold-tuned results. Threshold tuning largely removed the practical advantage of SMOTE, with the strongest tuned F1 values differing by less than 0.001.

Table 4.3: Threshold-Tuned Results

Version	No-SMOTE Model	No-SMOTE Threshold	No-SMOTE F1	SMOTE Model	SMOTE Threshold	SMOTE F1
v1	RandomForest	0.80	0.819141	CatBoost	0.44	0.818983
v2	XGBoost	0.83	0.818759	XGBoost	0.43	0.818828
v3	XGBoost	0.82	0.861660	XGBoost	0.51	0.861611
v4	DecisionTree	0.76	0.861826	XGBoost	0.47	0.861443
v5	RandomForest	0.80	0.862030	XGBoost	0.45	0.861520

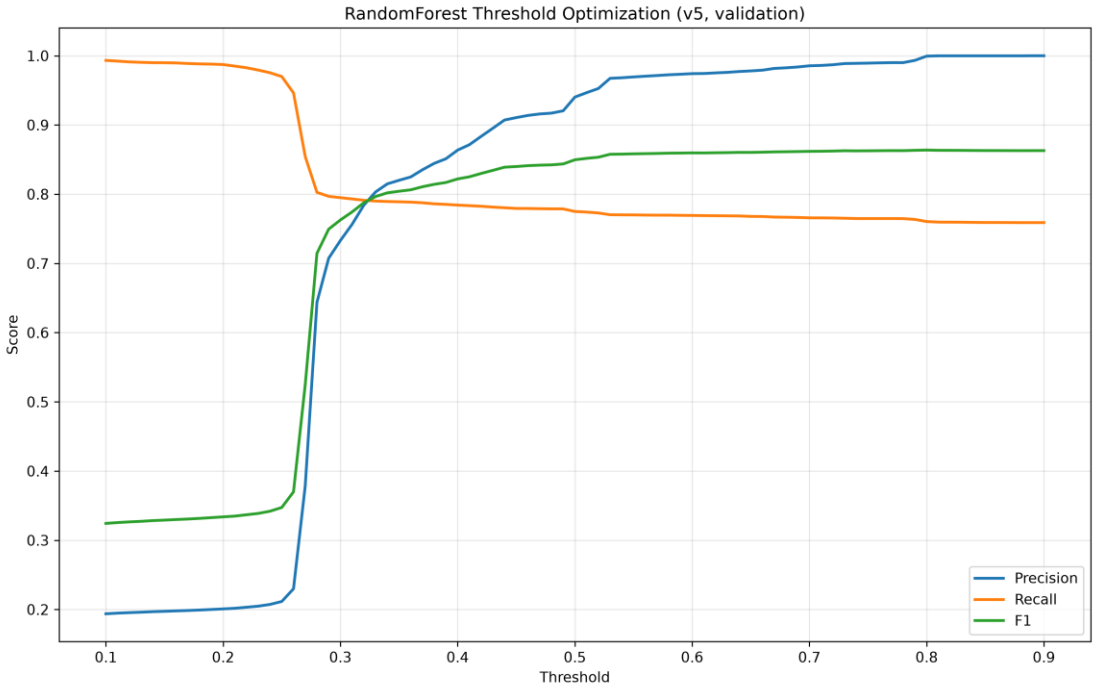

Figure 4.6: Threshold optimization curve for the v5 No-SMOTE Random Forest model.

4.10 Best Model Analysis

The best threshold-tuned operating point was v5 No-SMOTE Random Forest at threshold 0.80, with precision 0.999156, recall 0.758000, and fraud-class F1 of 0.862030. This value was only marginally higher than other strong tuned results.

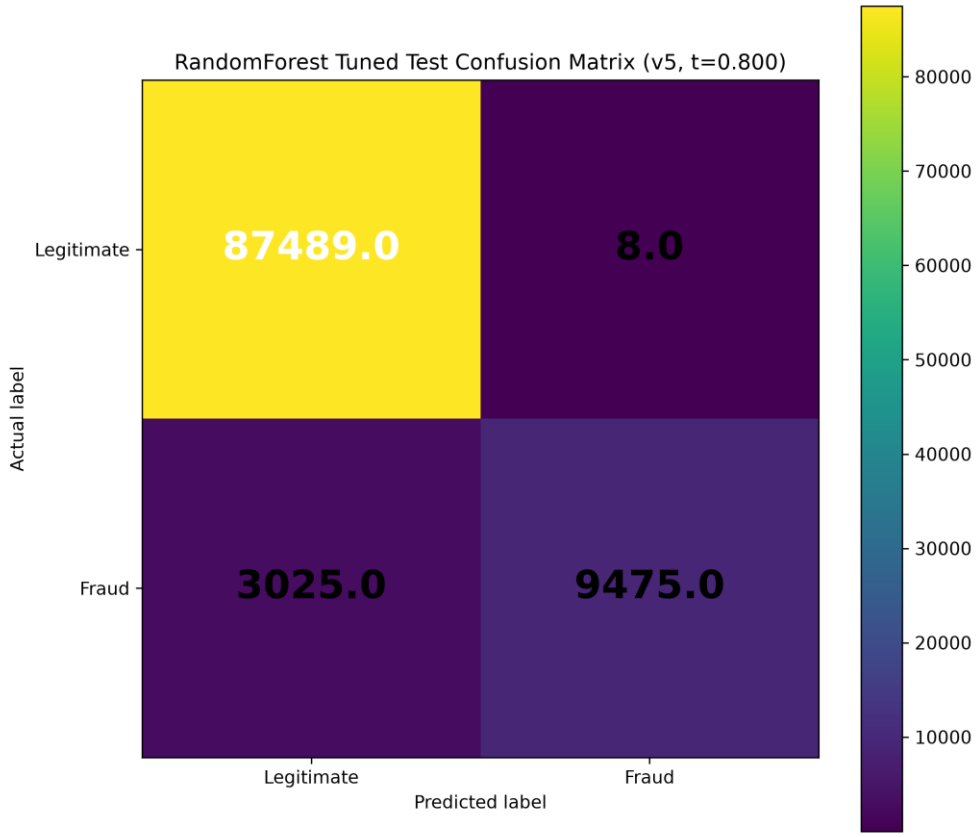

Figure 4.7: Tuned test confusion matrix for the v5 No-SMOTE Random Forest model at threshold 0.80.

4.11 Feature Importance Analysis

The Random Forest feature-importance output suggested that amount-category interaction and merchant-related indicators were influential in the selected v5 No-SMOTE model. These importance values are model-specific and should not be interpreted as causal effects.

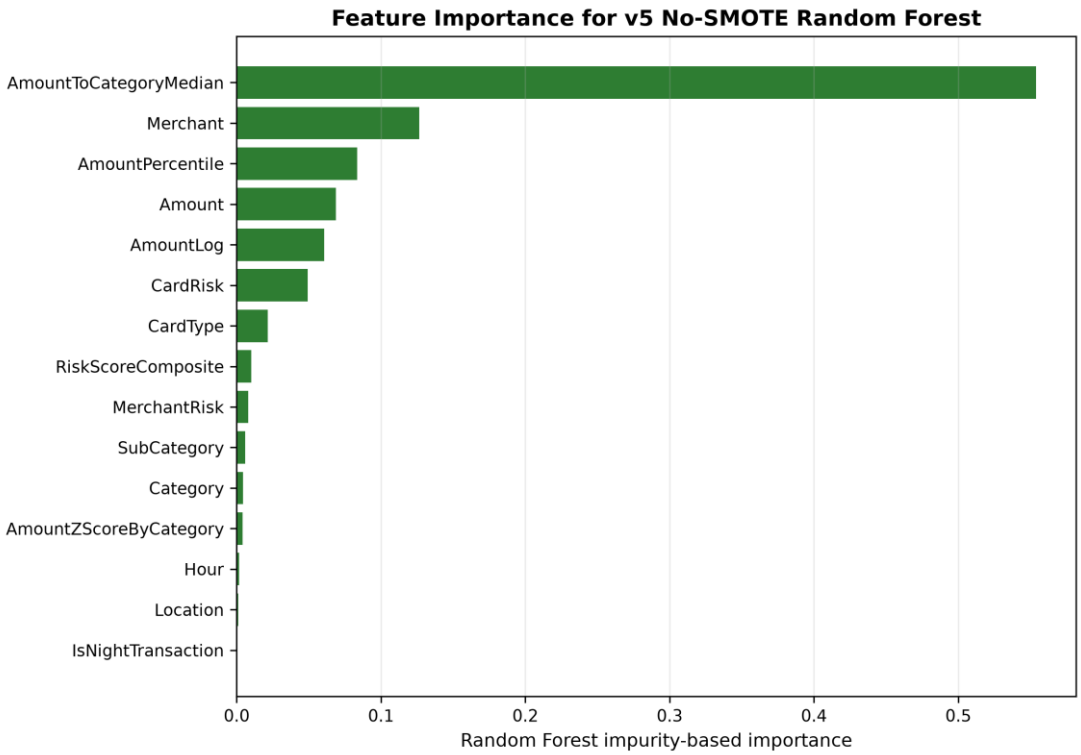

Figure 4.8: Feature importance for the selected v5 No-SMOTE Random Forest model.

4.12 Discussion of Findings

The results show that feature engineering was a major driver of model performance. Tree-based models dominated the strongest results because they can capture nonlinear interactions in heterogeneous tabular data. The SMOTE comparison also showed that resampling conclusions depend on the threshold used for classification.

4.13 Answering the Research Questions

First, progressive feature engineering improved fraud-class performance, especially from v2 to v3. Second, SMOTE improved default-threshold fraud-class F1. Third, threshold optimization made No-SMOTE and SMOTE results practically close. Fourth, Random Forest, CatBoost, and XGBoost dominated the strongest results. Fifth, interpretation is limited by synthetic data, proxy risk indicators, and absence of external validation.

4.14 Summary of Findings

Table 4.4: Key Findings Summary

Finding	Evidence	Interpretation
Progressive feature engineering improves fraud-class performance	v3-v5 outperform v1-v2 in fraud-class F1	Feature enrichment changes minority-class separability.
SMOTE improves default-threshold fraud-class F1	SMOTE selected models outperform no-SMOTE selected models in all five versions	The benefit is threshold-dependent.
Threshold tuning removes most practical SMOTE advantage	Best tuned F1 values differ by less than 0.001 across setups	Operating-point selection is central to deployment interpretation.
Tree-based methods dominate	Random Forest, XGBoost, CatBoost, and Decision Tree are selected in final best-model summaries	Nonlinear tabular interactions are important in this dataset.
Logistic Regression underperforms	No final best model is Logistic Regression	Linear baseline is less competitive for the engineered feature space.
Strongest versions are practically close	v3-v5 default and tuned results are close	Do not overstate the small differences among v3-v5.
External validity is limited	No external validation dataset is present	Results apply to the synthetic transaction dataset only.

Overall, the evidence supports a cautious conclusion that a reproducible tabular pipeline with progressive feature engineering can improve fraud-class detection on this synthetic dataset.

CHAPTER FIVE: CONCLUSION AND RECOMMENDATIONS

5.1 Summary of the Study

The study prepared a cleaned synthetic credit card transaction dataset, generated five progressively enriched dataset versions, trained five model families, compared No-SMOTE and SMOTE settings, and evaluated default-threshold and threshold-tuned performance.

5.2 Conclusion

Progressive feature engineering improved fraud-class performance on the synthetic transaction dataset. The strongest default-threshold result was v3 SMOTE Random Forest, while the strongest threshold-tuned operating point was v5 No-SMOTE Random Forest. However, the strongest v3-v5 results were practically close, and the findings should be interpreted within the limitations of the dataset and experimental design.

5.3 Contributions of the Study

1. A cleaned and reproducible master dataset workflow for fraud detection research.
2. A five-version feature engineering design using the same transaction population.
3. A controlled comparison of No-SMOTE and SMOTE training settings.
4. A threshold-tuned evaluation showing the importance of operating-point selection.
5. A transparent discussion of leakage-risk features and external-validity limits.

5.4 Recommendations

Future project work should add repeated-seed experiments, confidence intervals, and independent validation data. It should also document the construction and availability of MerchantRisk and CardRisk before considering those indicators for practical use.

5.5 Limitations Revisited

The main limitations remain the synthetic dataset, absence of external validation, possible proxy-feature leakage, static offline split, and lack of statistical uncertainty estimates.

5.6 Suggestions for Future Research

Future research should evaluate the pipeline on independent transaction datasets, compare additional imbalance strategies, test cost-sensitive thresholds, and apply robust interpretability methods beyond impurity-based feature importance.

REFERENCES

[1] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," Statistical Science, vol. 17, no. 3, pp. 235-255, 2002, doi: 10.1214/ss/1042727940.

[2] E. W. T. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun, "The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature," Decision Support Systems, vol. 50, no. 3, pp. 559-569, 2011, doi: 10.1016/j.dss.2010.08.006.

[3] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," Decision Support Systems, vol. 50, no. 3, pp. 602-613, 2011, doi: 10.1016/j.dss.2010.08.008.

[4] C. Whitrow, D. J. Hand, P. Juszczak, D. Weston, and N. M. Adams, "Transaction aggregation as a strategy for credit card fraud detection," Data Mining and Knowledge Discovery, vol. 18, no. 1, pp. 30-55, 2009, doi: 10.1007/s10618-008-0116-z.

[5] A. C. Bahnsen, D. Aouada, A. Stojanovic, and B. Ottersten, "Feature engineering strategies for credit card fraud detection," Expert Systems with Applications, vol. 51, pp. 134-142, 2016, doi: 10.1016/j.eswa.2015.12.030.

[6] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in Proc. IEEE Symposium Series on Computational Intelligence, 2015.

[7] J. Jurgovsky et al., "Sequence classification for credit-card fraud detection," Expert Systems with Applications, vol. 100, pp. 234-245, 2018, doi: 10.1016/j.eswa.2018.01.037.

[8] C. Phua, V. Lee, K. Smith, and R. Gayler, "A comprehensive survey of data mining-based fraud detection research," arXiv:1009.6119, 2010.

[9] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.

[10] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263-1284, 2009, doi: 10.1109/TKDE.2008.239.

[11] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," Intelligent Data Analysis, vol. 6, no. 5, pp. 429-449, 2002.

[12] M. Galar, A. Fernandez, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," IEEE Transactions on Systems, Man, and Cybernetics, Part C, vol. 42, no. 4, pp. 463-484, 2012, doi: 10.1109/TSMCC.2011.2161285.

[13] P. Branco, L. Torgo, and R. P. Ribeiro, "A survey of predictive modeling on imbalanced domains," ACM Computing Surveys, vol. 49, no. 2, pp. 1-50, 2016, doi: 10.1145/2907070.

[14] C. Elkan, "The foundations of cost-sensitive learning," in Proc. International Joint Conference on Artificial Intelligence, 2001, pp. 973-978.

[15] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in Proc. International Conference on Machine Learning, 2006, pp. 233-240, doi: 10.1145/1143844.1143874.

[16] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLOS ONE, vol. 10, no. 3, 2015, doi: 10.1371/journal.pone.0118432.

[17] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861-874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[18] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.

[19] J. R. Quinlan, "Induction of decision trees," Machine Learning, vol. 1, no. 1, pp. 81-106, 1986, doi: 10.1007/BF00116251.

[20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785-794, doi: 10.1145/2939672.2939785.

[21] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in Advances in Neural Information Processing Systems, 2018.

[22] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[23] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," Journal of Machine Learning Research, vol. 13, pp. 281-305, 2012.

[24] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in Proc. International Joint Conference on Artificial Intelligence, 1995, pp. 1137-1145.

[25] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," Journal of Machine Learning Research, vol. 11, pp. 2079-2107, 2010.

[26] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," BMC Bioinformatics, vol. 7, article 91, 2006, doi: 10.1186/1471-2105-7-91.

[27] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, pp. 1-21, 2012, doi: 10.1145/2382577.2382579.

[28] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, 2023, doi: 10.1016/j.patter.2023.100804.

[29] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," Journal of Machine Learning Research, vol. 3, pp. 1157-1182, 2003.

[30] M. Kuhn and K. Johnson, Applied Predictive Modeling. New York, NY, USA: Springer, 2013.

APPENDICES

Appendix A: Dataset Audit Summary

The final master dataset contained 499,985 unique records after removing exact duplicate rows from the raw merged provenance file. The final datasets had zero missing values, zero exact duplicate rows, and zero duplicate TransactionID values.

Appendix B: Experimental Configuration

The experiment used stratified 60/20/20 splitting, 5-fold cross-validation on the training split, PR-AUC tuning, validation-based model selection, untouched test reporting, and threshold optimization based on validation predictions.

Appendix C: Full Result Summary

The selected No-SMOTE, SMOTE, and threshold-tuned results are summarized in Tables 4.1 through 4.3. The strongest default-threshold result was v3 SMOTE Random Forest, and the strongest tuned operating point was v5 No-SMOTE Random Forest.

Appendix D: Reproducibility Notes

The final datasets, result summaries, figures, and audit reports were generated from the active research-clean repository evidence. Legacy scripts and stale result folders were excluded from the final methodology.

Appendix E: Supervisor Review Notes

This section is reserved for comments, corrections, and guidance from the supervisor during review.